

CDP优化方案 V3.0-02

OneID 改造 与全链路优化

水平扩展 · 多对象筛选 · 自然语言 Segment

2026 年 6 月

合并来源：CDP 专项优化 + CDP Agent

1. 方案总览

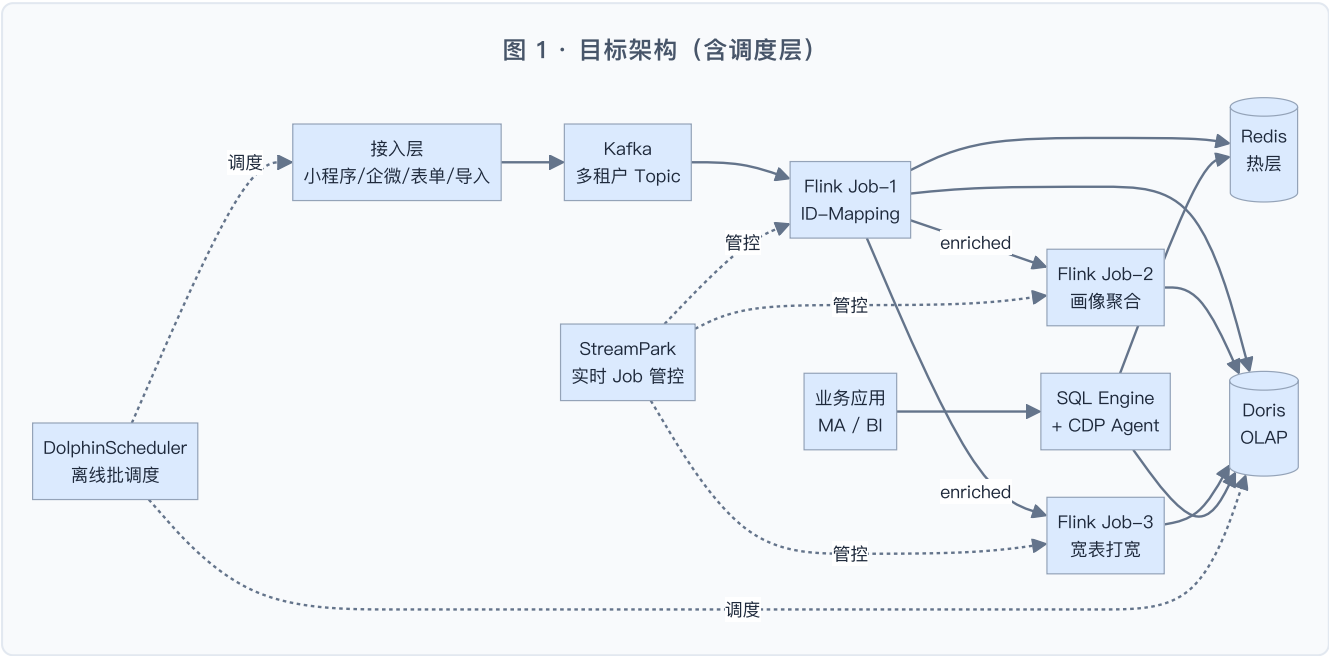
核心目标

- ▶ OneID 实时化：Java 同步服务 → Flink + Redis，识别延迟 P50 < 5ms
- ▶ 多租户水平扩展：Kafka / Flink / Redis / Doris 按租户分级伸缩
- ▶ 多对象筛选：User 单实体 → 人-货-场-线索跨对象圈选（长期）
- ▶ NL Segment：自然语言 → 候选规则 → 引擎校验 → 人工确认

技术组件

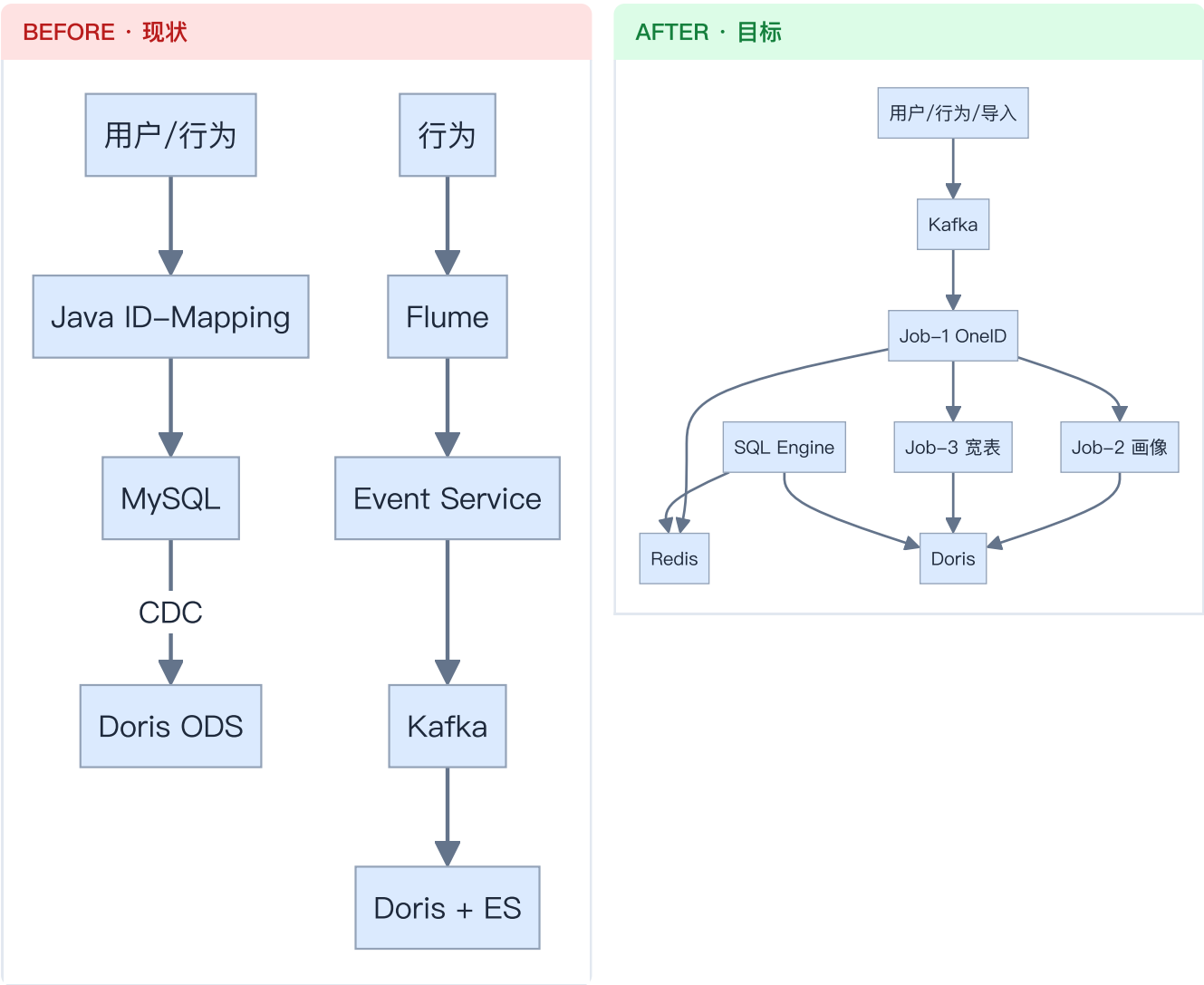
Kafka 多租户事件总线	Flink 实时 OneID / 画像 / 宽表	Redis OneID 热层缓存	Doris OLAP 画像 / 圈选
StreamPark Flink 实时 Job 管控	DolphinScheduler 离线 / 批任务调度	SQL Engine 模板化 OLAP 查询	CDP Agent NL 圈人编排层

StreamPark 负责实时 Flink Job 启停与扩缩；DolphinScheduler 负责离线导入、批量标签计算、数据同步、Doris 运维及 Job 依赖编排。



2. 短期目标：OneID 改造与水平扩展

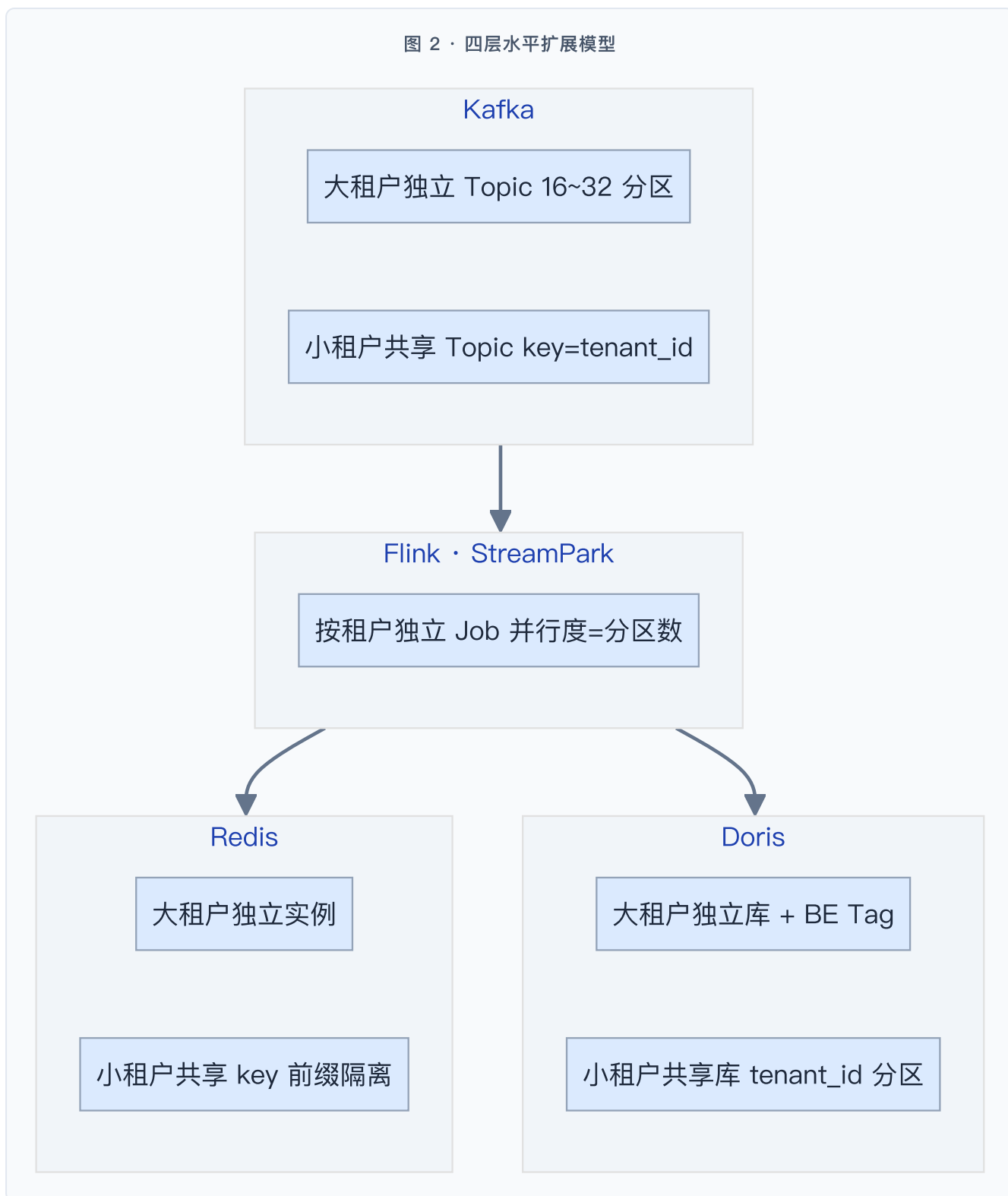
2.1 链路 Before / After



维度	Before	After
OneID 识别	Java 同步, 10–100ms	Flink + Redis, < 5ms
数据写入	MySQL → CDC → Doris	Kafka → Flink 直写 Doris
行为链路	Flume + Event Service + ES	统一 UserEvent, 无 ES
横向扩展	IDM Pod + nginx 路由	分区 + 并行度 + 消费组
任务调度	分散 Cron / 手工	DolphinScheduler 统一编排

2.2 多租户四级扩展

图 2 · 四层水平扩展模型



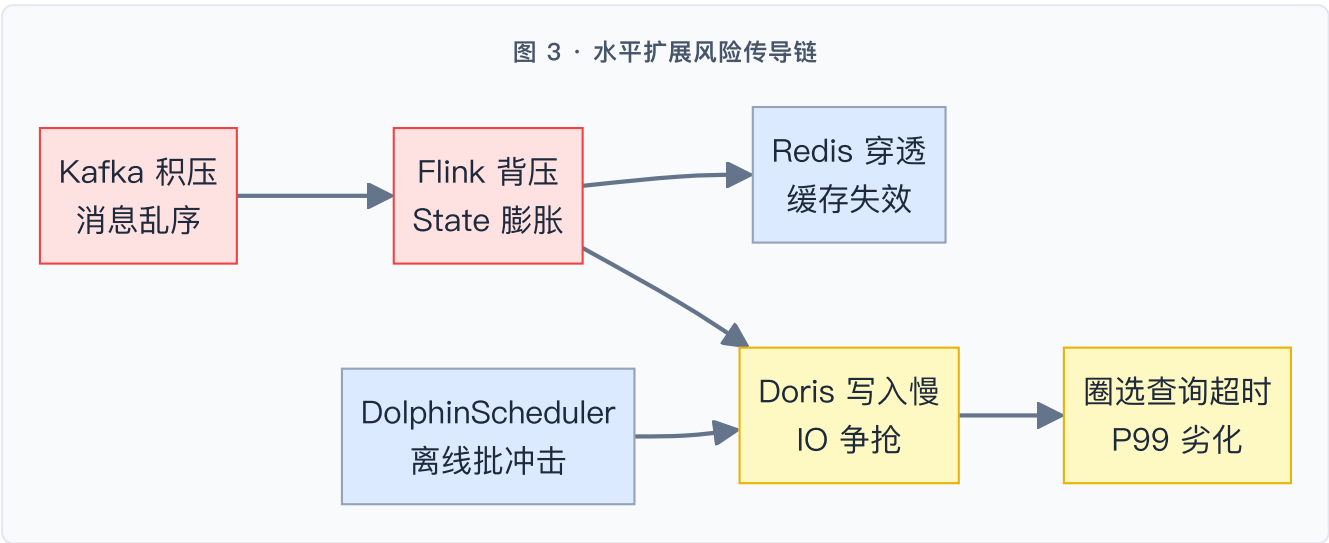
规模	Kafka	分区	Redis	Flink 副本	Doris BE
dev (<1万)	1	4	1	1	1
medium (千万)	2	8	2	2	4
large (1亿)	3	16	3	4	8
xlarge (2亿)	5	32	6	8	12

2.3 水平扩展带来的问题与对策

扩容不是简单加节点，以下四类问题是主要风险点：

组件	典型问题	风险	对策
Redis	① 大租户热 Key 打满单分片 ② merge 后多副本缓存不一致 ③ 扩容后冷数据穿透 Doris ④ LRU 淘汰导致命中率骤降	高	大租户独立 Redis Cluster；merge 后主动失效 channel key； 写时双写 + 短 TTL 兜底；监控命中率 < 90% 告警
Kafka	① 共享 Topic 仅保证同 tenant 有序 ② create/merge 事件跨分区乱序 ③ 扩分区后 rebalance 重复消费 ④ link_keys 跨分区关联竞态	高	merge 相关事件 key 统一为 one_id； 幂等写入 + merge_log 去重； rebalance 期间开启幂等检查窗口
Flink	① Doris 写入慢 → 背压 → Kafka 积压 ② keyBy 热点 subtask 倾斜 ③ RocksDB State 过大 checkpoint 超时 ④ 三 Job 链路延迟叠加	高	Async Sink + 批量写入 Doris； 热点 key 加盐再聚合； State TTL + 增量 checkpoint； 各 Job 独立监控 lag，超 60s 告警
Doris	① 小租户共享库全表扫描 ② 分桶倾斜导致长尾查询 ③ 实时写入与圈选查询抢 IO ④ 宽表 JOIN 内存溢出	中	强制 tenant_id 分区裁剪； 按数据量重估 BUCKETS (≥16)； 读写分离：写入走 Routine Load，查询走 Follower； 圈选走 SQL Engine 模板 + 超时熔断
DolphinScheduler	① 离线批与实时流资源争抢 ② 多租户批任务并发打满 Doris ③ 任务依赖失败级联	中	离线任务限流 + 租户级配额； 批流分时调度（凌晨批 / 白天实时）； 关键路径设置重试 + 告警 + 人工兜底

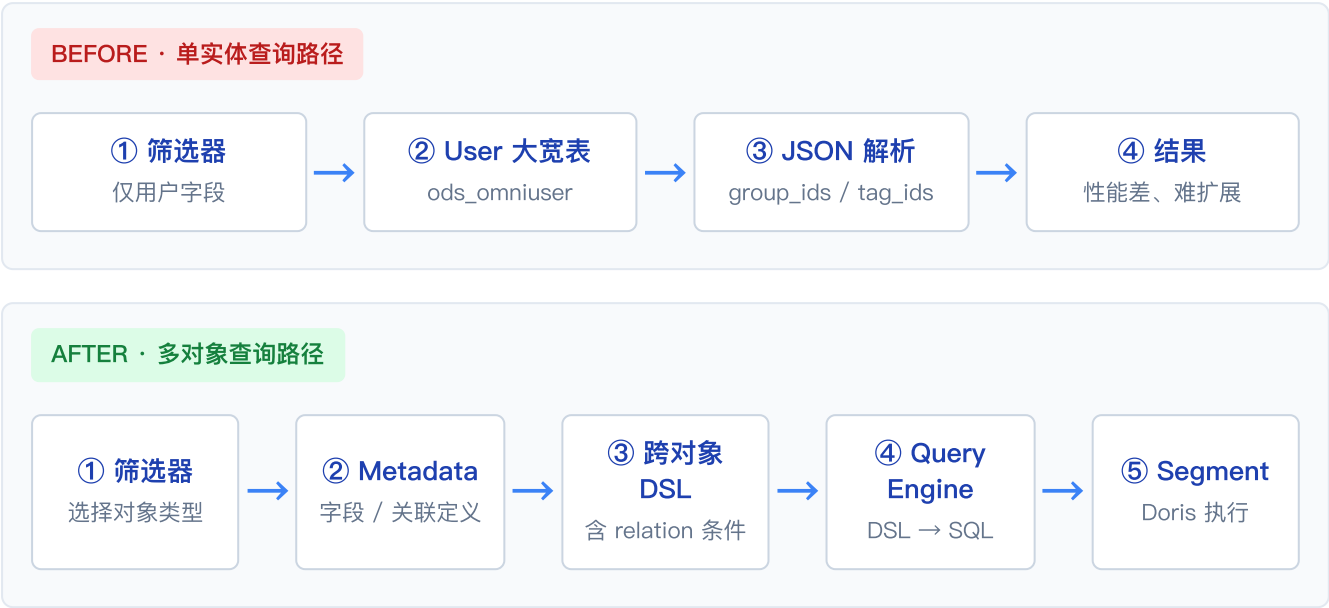
图 3 · 水平扩展风险传导链



3. 长期目标：多对象筛选

维度	Before	After
核心实体	仅 User (OneID)	User + Lead + Account + Product + Store
存储	单表大 JSON (group_ids / tag_ids)	对象独立表 + object_relations
筛选	用户属性 + 行为	跨对象 JOIN + 关联行为
竞品差异	与火山 VeCDP 持平	多对象圈选 (火山不支持)

3.1 查询路径对比 (图 4)



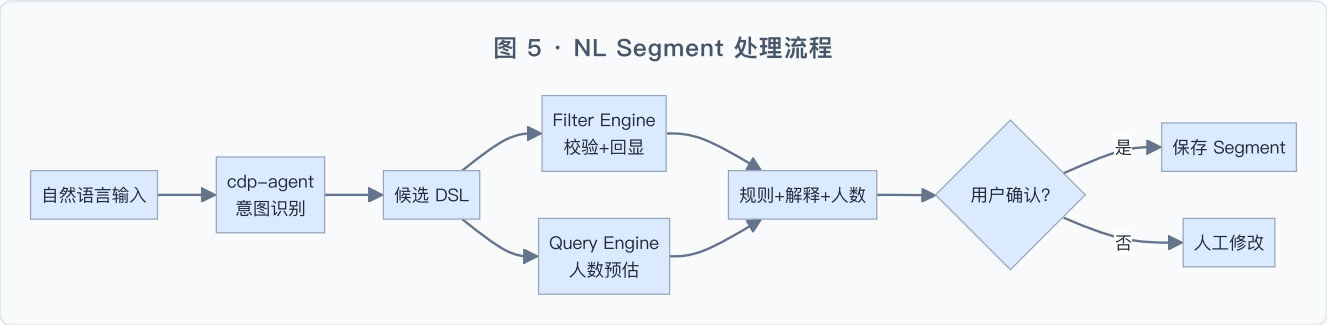
	User	Lead	Account	Product	Store
User	—	← belongs_to	owns →		visited →
Lead	belongs_to →	—			
Account			—	purchased →	
Product				—	
Store	← visited				—

矩阵中蓝色单元格表示已定义的关联关系，存储于 `object_relations` 表，`Query Engine` 通过 `JOIN` 实现跨对象筛选。

跨对象圈选示例：「地址在上海、公司规模 > 500 的线索，且关联用户带有 VIP 标签」
→ DSL `object=lead + relation.belongs_to(user.tags contains vip)` → Doris JOIN 执行

4. 自然语言 Segment (CDP Agent)

- ▶ LLM 只负责意图理解，不直接拼 SQL、不直接发规则
- ▶ 候选 DSL 必须经过 Filter / Query Engine 校验 + 人数预估
- ▶ 用户确认后才走现有规则保存链路



阶段	能力	周期
Phase 1	NL 圈人 + 人数预估 + 多轮澄清 + 只读 MCP	约 2 个月
Phase 2	跨对象 NL 圈人 + 规则体检 + 匹配诊断	约 2 个月
Phase 3	Doris Hybrid Search + 选项/文本语义召回	约 2 个月